

Online Shopping Database

Introduction:

Databases are critical for storing structured data, especially in online shopping systems where large amounts of customer, order, and payment information must be efficiently kept and retrieved. In this work, I was in charge of building and implementing a relational database for an online shopping website. The goal was to import data from CSV files into organized tables, create associations between them, and run various SQL queries to extract useful insights.

The project requires a thorough understanding of database normalization, data integrity, and query optimization. My method was structured in the following way:

- Database design entails creating a well-structured database with suitable primary and foreign key restrictions.
- Data Import and Validation-- Importing CSV data, verifying accuracy, and validating relationships.
- Query Development-- Creating SQL queries to address specific business issues, and enhance data retrieval.
- Performance considerations include ensuring queries are efficient and optimizing them through correct joins.

PART 1: Database creation and Data import

To begin, I established a database called OnlineShoppingDB and defined five tables: Customers, Products, Orders, Order_Items, and Payment. Each table was designed with specific data types and limitations to ensure data integrity.

- **Creating database**
 - Creating database
 - Using the created database

```
1  -- PART 1
2
3  -- Creating Database
4  Create database OnlineShoppingDB;
5
6  -- Using Database
7  Use OnlineShoppingDB;
8
```

- Creating tables according to the CSV file:
 - Customers-- Customer.csv file has 120 records and provides the customer_id, name, email, phone, and country. Customer_id is a unique key identifier.
 - Products-- The Product.csv file provides details of the 5 different products that can be purchased. This includes the product_id,

product_name, category, and price. The product_id column provides a unique identifier for each product.

- Orders-- The Orders.csv file provides a total of 120 records with order_id, customer_id and order_date. The order_id column provides a unique identifier for each order.
- Order_items-- Order_items.csv lists the total 120 rows including order_item_id, order_id, product_id, quantity, price_each, Total_price and Total_amount. The order_item_id column provides a unique identifier for each order-item. Total_amount is calculated based on the multiple order_id placed for different products.
- Payments-- Payments.csv includes 120 rows including payment_id, order_id, payment_date, payment_method and Amount_paid. The payment_id column provides a unique identifier for this table.

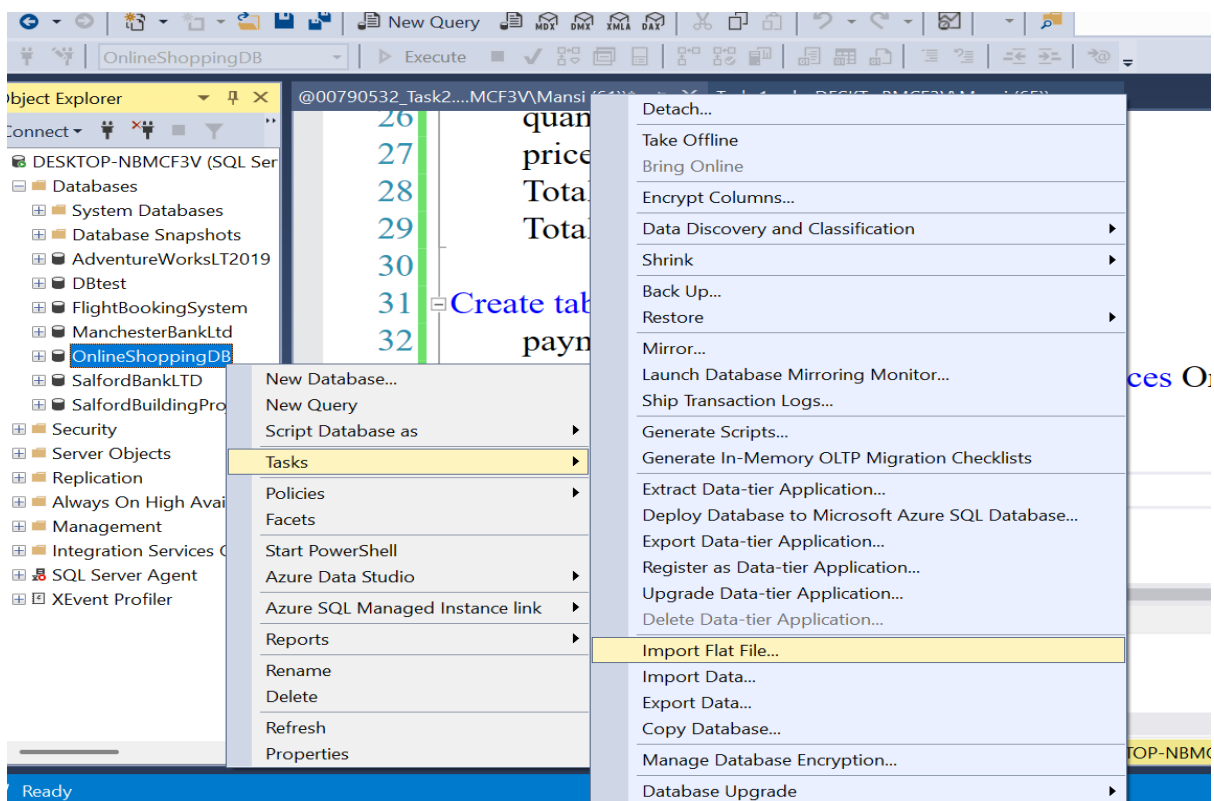
```
~
9  -- Creating Tables
10 Create table Customers(
11     customer_id int Primary key identity(1,1),
12     name nvarchar(200) not null,
13     email nvarchar(200) not null,
14     phone nvarchar(50) not null,
15     country nvarchar(100) not null);
16
17 Create table Products(
18     product_id int primary key identity(1,1),
19     product_name nvarchar(50) not null,
20     category nvarchar(50) not null,
21     price decimal(10,2));
22
23 Create table Orders(
24     order_id int Primary key not null,
25     customer_id int foreign key(customer_id) references Customers(customer_id),
26     order_date date not null);
27
28 Create table Order_items(
29     order_item_id int Primary key not null,
30     order_id int foreign key(order_id) references Orders(order_id),
31     product_id int foreign key(product_id) references Products(product_id),
32     quantity int not null,
33     price_each decimal(10,2),
34     Total_price decimal(10,2),
35     Total_amount decimal(10,2));
36
37 Create table Payments(
38     payment_id int Primary key not null,
39     order_id int foreign key(order_id) references Orders(order_id),
40     payment_date date not null,
41     payment_method nvarchar(50) not null,
42     Amount_paid decimal(10,2));
43
```

- **Normalization and Relationship**

- The database is normalized to the third normal form (3NF) to reduce redundancy and improve query performance. The relationships between tables are established as follows:
 - Customers and Orders-- A one-to-many relationship (a single client may submit numerous orders).
 - Orders and Order_Items-- A one-to-many relationship (a single order may contain numerous products).
 - Products and Order_Items-- A one-to-many relationship (a single product can appear in several orders).
 - Orders and Payments-- A one-to-many link (a single order has numerous payment records).

- **Data Import**

- Loading CSV Data-- The CSV files were loaded into temporary tables to avoid direct mistakes in the main database tables.
- Inserting Data into Tables-- Data was transferred from temporary tables to final structured tables, guaranteeing consistency within set restrictions.
- Verifying Data Integrity-- Following import, queries were run to ensure all entries were correctly inserted.
- Entity-relationship diagram
- Removing Temporary Tables-- Temporary tables were removed once the data had been successfully migrated.



Import Flat File 'OnlineShoppingDB'

Specify Input File

Introduction

Specify Input File

Preview Data

Modify Columns

Summary

Results

Help

Specify Input File

This operation will create a table from your input file.

Location of file to be imported

C:\mans\University\Trimester_2\ADB\Dataset for Task 2\Dataset for Task 2\Custom...

Browse...

New table name:

Customers

Table schema:

dbo

< Previous

Next >

Cancel

Import Flat File 'OnlineShoppingDB'

Preview Data

Introduction

Specify Input File

Preview Data

Modify Columns

Summary

Results

Help

Preview Data

This operation analyzed the input file structure to generate the preview below for up to the first 50 rows.

customer_id	name	email	phone	country
1	Vanessa ...	martha6...	+1-988-...	Germany
2	Andrew ...	codypark...	6000415...	Australia
3	Emma D...	kaylame...	081-770-...	UK
4	Wendy S...	jerrybax...	+1-329-...	Canada
5	Heather ...	samuelca...	187.745...	UK
6	Theresa J...	kylepugh...	529.922...	Canada
7	Stacy Ha...	kenneth...	+1-229-...	Canada
8	Andre Be...	tranheath...	391-401-...	UK
9	Joy Gomez	james34...	+1-187-...	Australia
10	Heather ...	millerkell...	(248)296...	USA
11	Mark An...	christoph...	515-826-...	Canada
12	Thomas J...	hortonja...	(176)072...	UK
13	Kendra S...	davalis...	+1-983-...	USA
14	Matthew ...	hannahw...	+1-713-...	Germany
15	...	...	...	...

☒ Use Rich Data Type Detection - may provide a closer type fit. However, cells with anomalous values may be dropped.

< Previous

Next >

Cancel

Import Flat File 'OnlineShoppingDB'



Summary

[Introduction](#)
[Specify Input File](#)
[Preview Data](#)
[Modify Columns](#)
[Summary](#)
[Results](#)

Help

Summary

To complete the operation using the specified inputs, click Finish.

Import Information

- Name: DESKTOP-NBMCF3V
- Database Name: OnlineShoppingDB
- Table Name: dbo.Customer
- File to be imported: C:\mansil\University\Trimester_2\ADB\Dataset for Task 2\Dataset for Task 2\OnlineShoppingDB.csv

< Previous

Finish

Cancel

Import Flat File 'OnlineShoppingDB'



Modify Columns

[Introduction](#)
[Specify Input File](#)
[Preview Data](#)
[Modify Columns](#)
[Summary](#)
[Results](#)

Help

Modify Columns

This operation generated the following table schema. Please verify if schema is accurate, and if not, please make any changes.

Column Name	Data Type	Primary Key	☐ Allow Nulls	
customer_id	int	☒	☐	
name	nvarchar(200)	☐	☐	
email	nvarchar(200)	☐	☐	
phone	nvarchar(50)	☐	☐	
country	nvarchar(100)	☐	☐	

Row granularity of error reporting (performance impact with smaller ranges)

No Range

< Previous

Next >

Cancel

Import Flat File 'OnlineShoppingDB'

Modify Columns

[Introduction](#)
[Specify Input File](#)
[Preview Data](#)
[Modify Columns](#)
[Summary](#)
[Results](#)

[Help](#)

Modify Columns

This operation generated the following table schema. Please verify if schema is accurate, and if not, please make any changes.

Column Name	Data Type	Primary Key	☐ Allow Nulls	
order_id	int	☒	☐	
customer_id	int	☐	☐	
order_date	date	☐	☐	

Row granularity of error reporting (performance impact with smaller ranges) No Range

< Previous
Next >
Cancel

```

44 -- Inserting Record From Csv Files
45 Insert into Customers(name,email,phone,country)
46     select name, email, phone,country from dbo.Customer;
47
48 Insert into Orders(order_id,customer_id,order_date)
49     select order_id, customer_id,order_date from dbo.[Order];
50
51 Insert into Order_items(
52     order_item_id,order_id,product_id,quantity,price_each>Total_price>Total_amount)
53     select order_item_id,order_id,product_id,quantity,price_each>Total_price>Total_amount
54     from dbo.Order_item;
55
56 Insert into Products(product_name,category,price)
57     select product_name,category,price from dbo.[Product];
58
59 Insert into Payments(payment_id,payment_date,order_id,payment_method,Amount_paid)
60     select payment_id,payment_date,order_id,payment_method,Amount_paid from dbo.Payment;
61
62 -- Confirming uploaded data
63 select *from Customers;
64 select *from Order_items;
65 select *from Orders;
66 select *from Payments;
67 select *from Products;
68

```

83 %

Results Messages

	customer_id	name	email	phone	country
1	1	Vanessa Flores	martha65@howard.com	+1-988-515-3143	Germany
2	2	Andrew Rodriguez	codyparker@yahoo.com	6000415241	Australia
3	3	Emma Davis	kaylamedina@gmail.com	081-770-4641	UK
4	4	Wendy Smith	jerrybaxter@hotmail.com	+1-329-975-1701x875	Canada
5	5	Heather Brown	samuelcaldwell@mcdonald.com	187.745.4380	UK
6	6	Theresa Johnson	kylepugh@gmail.com	529.922.5882	Canada
7	7	Stacy Hansen	kennethmorris@gmail.com	+1-229-121-7653x561	Canada
8	8	Andre Beard	tranheather@maynard.org	391-401-6328x49981	UK
9	9	Joy Gomez	james34@russell-austin.com	+1-187-433-6068x72333	Australia
10	10	Heather Brown DVM	millerkelly@gmail.com	(248)296-9906x81251	USA
11	11	Mark Anderson	christophercox@yahoo.com	515-826-6139	Canada

83 %

Results Messages

	order_item_id	order_id	product_id	quantity	price_each	Total_price	Total_amount
1	1	186	4	1	50.00	50.00	4400.00
2	2	103	5	5	120.00	600.00	600.00
3	3	133	1	3	1000.00	3000.00	3600.00
4	4	138	5	4	120.00	480.00	1280.00
5	5	123	2	3	800.00	2400.00	2400.00
6	6	167	4	3	50.00	150.00	150.00
7	7	128	5	2	120.00	240.00	5240.00
8	8	136	1	4	1000.00	4000.00	5650.00
9	9	162	2	5	800.00	4000.00	7250.00
10	10	209	5	3	120.00	360.00	360.00
11	11	191	2	2	800.00	1600.00	6600.00

83 %

Results Messages

	order_id	customer_id	order_date
1	101	42	2024-04-19
2	102	63	2024-04-20
3	103	32	2024-03-23
4	104	35	2024-03-21
5	105	91	2024-02-26
6	106	28	2024-04-26
7	107	120	2024-02-18
8	108	66	2024-01-30
9	109	96	2024-03-22
10	110	108	2024-03-04
11	111	10	2024-02-29

83 %

Results Messages

	payment_id	order_id	payment_date	payment_method	Amount_paid
1	201	163	2025-01-01	Bank Transfer	0.00
2	202	180	2025-01-01	Bank Transfer	700.00
3	203	111	2025-01-01	PayPal	0.00
4	204	106	2025-01-02	PayPal	1000.00
5	205	180	2025-01-01	Bank Transfer	700.00
6	206	205	2025-01-02	Bank Transfer	300.00
7	207	155	2025-01-02	Credit Card	450.00
8	208	147	2025-01-02	PayPal	0.00
9	209	116	2025-01-01	PayPal	0.00
10	210	103	2025-01-02	PayPal	600.00
11	211	199	2025-01-02	Bank Transfer	0.00

83 %

Results Messages

	product_id	product_name	category	price
1	1	Laptop	Electronics	1000.00
2	2	Smartphone	Electronics	800.00
3	3	Headphones	Accessories	150.00
4	4	Keyboard	Accessories	50.00
5	5	Coffee Maker	Appliances	120.00

68

69 -- Dropping unwanted tables

70 drop table Customer;

71 drop table Order_item;

72 drop table [Order];

73 drop table [Product];

74 drop table Payment;

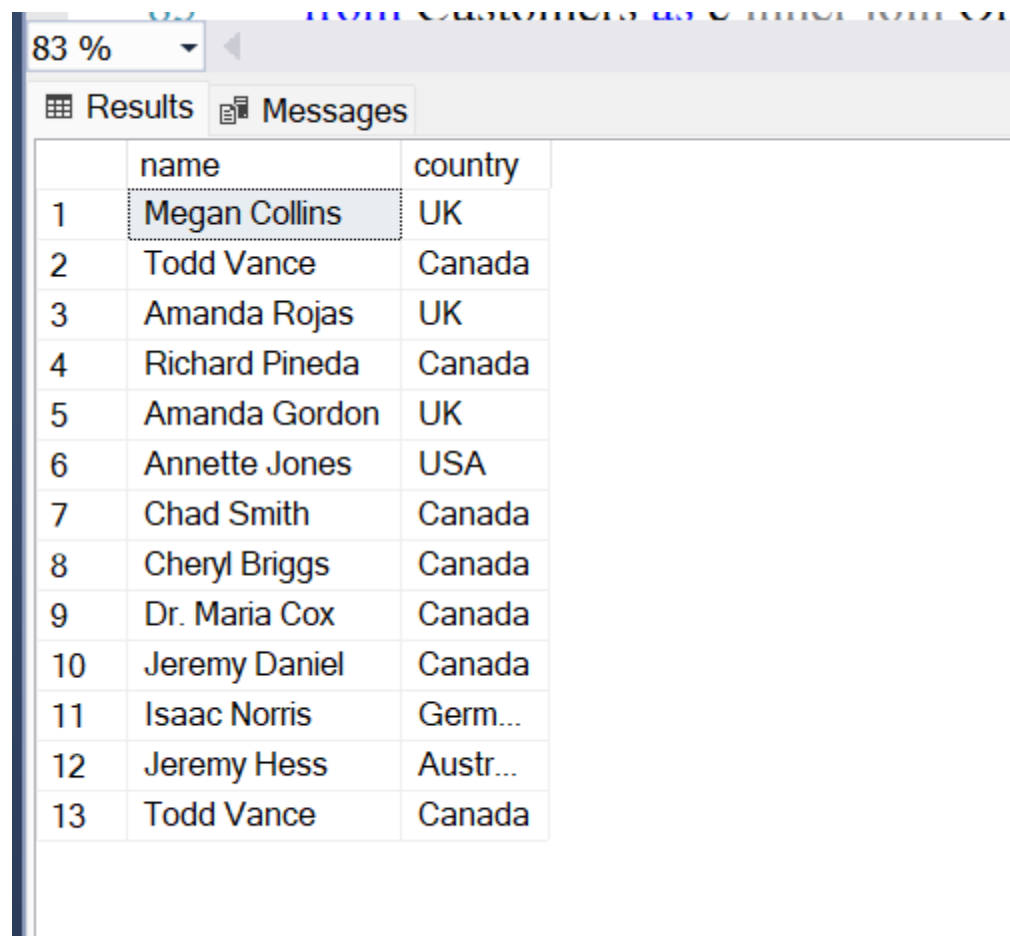
75

Part 2: Query Solutions

Ques 2: Write a query that returns the names and countries of customers who made orders with a total amount between £500 and £1000.

Solution: To find clients whose total order amounts are within this range, the query aggregated order totals by customer. A join was performed on Customers, Orders, and Order_Items, grouping results by customer and using a HAVING filter to pick only those inside the required range. This information enables organizations to identify mid-tier customers for focused marketing strategies.

```
77 -- PART 2
78
79 -- QUES 2:
80 /* Write a query that returns the names and countries of customers who made orders with
81 a total amount between £500 and £1000. */
82 Select c.name,c.country
83 from Customers as c inner join Orders as o on c.customer_id=o.customer_id
84 inner join Order_items as oi on o.order_id=oi.order_id
85 group by c.name,c.country,oi.Total_amount
86 having 500< oi.Total_amount and oi.Total_amount<1000 order by oi.Total_amount;
87
```



	name	country
1	Megan Collins	UK
2	Todd Vance	Canada
3	Amanda Rojas	UK
4	Richard Pineda	Canada
5	Amanda Gordon	UK
6	Annette Jones	USA
7	Chad Smith	Canada
8	Cheryl Briggs	Canada
9	Dr. Maria Cox	Canada
10	Jeremy Daniel	Canada
11	Isaac Norris	Germ...
12	Jeremy Hess	Austr...
13	Todd Vance	Canada

Ques 3: Get the total amount paid by customers belonging to UK who bought at least more than three products in an order.

Solution: This query searched for UK-based consumers and identified payments when the total product count in an order exceeded three. The use of GROUP BY and HAVING ensured that only relevant transactions were included. This data aids in studying purchasing patterns among high-volume buyers.

```
--
88 -- QUES 3:
89 /* Get the total amount paid by customers belonging to UK who bought at least more than
90 three products in an order. */
91 Select c.customer_id,c.name,sum(p.amount_paid) as Total_amount_paid
92 from Customers c inner join Orders o on c.customer_id=o.customer_id
93 inner join Order_items oi on o.order_id=oi.order_id
94 inner join Payments p on oi.order_id=p.order_id
95 where c.country='UK'
96 group by c.customer_id,c.name
97 having sum(oi.quantity)>3;
```

Results		Messages	
	customer_id	name	Total_amount_paid
1	16	Larry Chapman	133760.00
2	23	Lisa Thomas	34890.00
3	32	Megan Collins	1800.00
4	41	April Campbell	6800.00
5	42	Paula Wilson	118470.00
6	85	Christopher Davis	43360.00
7	86	Robert Keller	300.00
8	95	Jose Baker	600.00
9	100	Julia Bailey	4000.00
10	108	Amanda Rojas	2250.00
11	118	Carla Patterson	450.00

Ques 4: Write a query that returns the highest and second highest amount_paid from UK or Australia – this is calculated after applying VAT as 12.2% multiplied by the amount_paid. Some of the results are not integer values and your client has asked you to round the result to the nearest integer value.

Solution: The amount_paid field received a percentage-based VAT adjustment (12.2%), with values formatted to the nearest integer using ROUND(). ORDER BY DESC and TOP 2 yielded the highest two payments. This query analyzes large-ticket sales in important regions, which is useful for revenue tracking.

```

100 -- QUES 4:
101 /* Write a query that returns the highest and second highest amount_paid from UK or
102 Australia – this is calculated after applying VAT as 12.2% multiplied by the amount_paid.
103 Some of the results are not integer values and your client has asked you to round the
104 result to the nearest integer value. */
105 Select top 2 p.order_id, round((sum(p.Amount_paid)*1.122),0) as Total_amount_paid
106 from Customers c join Orders o on o.customer_id=c.customer_id
107 inner join Payments p on o.order_id=p.order_id
108 where c.country in ('UK','Australia' )
109 group by o.customer_id,p.order_id
110 order by Total_amount_paid desc

```

%
Results Messages

order_id	Total_amount_paid
101	44128.00000
191	42210.00000

Ques 5: Write a query that returns a list of the distinct product_name and the total quantity purchased for each product called as total_quantity. Sort by total_quantity.

Solution: To find the most popular products, the total quantity purchased was added for each product, with GROUP BY and ORDER BY DESC guaranteeing that often-purchased items appeared first. This information is essential for inventory management and demand forecasting.

```

111
112 -- QUES 5:
113 /* Write a query that returns a list of the distinct product_name and the total quantity
114 purchased for each product called as total_quantity. Sort by total_quantity. */
115 Select p.product_name, sum(oi.quantity) as total_quantity
116 from Products p inner join Order_items oi on p.product_id=oi.product_id
117 group by p.product_name,p.product_id
118 order by total_quantity
119

```

83 %
Results Messages

	product_name	total_quantity
1	Coffee Maker	53
2	Headphones	63
3	Smartphone	75
4	Laptop	83
5	Keyboard	91

Ques 6: Write a stored procedure for the query given as: Update the amount_paid of customers who purchased either laptop or smartphone as products and amount_paid>=£17000 of all orders to the discount of 5%.

Solution: A stored procedure was developed to add a 5% discount to payments for orders over £17,000 that included either a laptop or a smartphone. The UPDATE statement with a subquery found eligible orders. A backup table was also created to preserve the original payment information before applying the discount. This stored method guarantees that promotional discounts are automatically applied when specific requirements are met.

```

117
120 -- QUES 6:
121 /* Write a stored procedure for the query given as: Update the amount_paid of customers
122 who purchased either laptop or smartphone as products and amount_paid >= £17000 of
123 all orders to the discount of 5%. */
124
125 -- Creating a backup table if it doesn't exist (preventing data loss)
126 If not exists (select * from INFORMATION_SCHEMA.TABLES
127 where TABLE_NAME = 'Payments_Backup')
128 Begin
129 Select * into Payments_Backup FROM Payments;
130 End
131
144 -- Creating procedure for discount
145 Create procedure ApplyDiscount
146 As
147 Begin
148 update Payments
149 set Amount_paid = Amount_paid * 0.95
150 where order_id in (
151 SELECT oi.order_id FROM Order_Items oi
152 JOIN Products pr ON oi.product_id = pr.product_id
153 WHERE pr.product_name IN ('Laptop', 'Smartphone')
154 GROUP BY oi.order_id
155 HAVING (
156 select SUM(p.amount_paid) from Payments p where p.order_id = oi.order_id
157 >= 17000)
158 end
159
160 exec ApplyDiscount

```

```

132 -- checking columns
133 SELECT p.order_id, SUM(p.amount_paid) as total_amount,
134 (sum(p.Amount_paid) * 0.95) as discounted_amount
135 FROM Payments p JOIN Orders o ON p.order_id = o.order_id
136 WHERE p.order_id IN (
137 SELECT DISTINCT oi.order_id
138 FROM Order_Items oi
139 JOIN Products pr ON oi.product_id = pr.product_id
140 WHERE pr.product_name IN ('Laptop', 'Smartphone') )
141 GROUP BY p.order_id, p.Amount_paid
142 HAVING SUM(p.amount_paid) >= 17000 order by p.order_id
143
144 -- Creating procedure for discount

```

33 %

Results Messages

	order_id	total_amount	discounted_amount
1	101	39330.00	37363.5000
2	165	34200.00	32490.0000
3	175	19000.00	18050.0000
4	186	33440.00	31768.0000
5	191	37620.00	35739.0000

Ques 7: Additional Queries

Solution: To demonstrate SQL expertise, six more queries were written:

- Total Payments per Payment Method
 - Allows businesses to determine which payment methods are most commonly utilized.

```
163 -- QUES 7: Extra ques
164
165 -- USING PAYMENTS_BACKUP FOR ORIGINAL DATA
166 -- 1. Find the total amount paid per payment method
167 Select payment_method,sum(Amount_paid) as total_amount_paid from Payments_Backup
168   group by payment_method order by total_amount_paid desc
```

payment_method	total_amount_paid
1 Bank Transfer	113680.00
2 Credit Card	99100.00
3 Debit Card	82570.00
4 PayPal	43300.00

- Customers who have ordered at least one product
 - EXISTS is used to identify current customers.

```
170 -- 2. Customers who ordered from at least 1 category
171 Select c.customer_id,c.[name],c.email,c.country
172   from Customers c
173  where exists (
174    select 1,p.product_name from Orders o join Order_items oi on o.order_id=oi.order_id
175    join Products p on oi.product_id=p.product_id
176    where c.customer_id=o.customer_id)
177
```

customer_id	name	email	country
1 2	Andrew Rodriguez	codyparker@yahoo.com	Australia
2 3	Emma Davis	kayamedina@gmail.com	UK
3 5	Heather Brown	samuelsdwell@mcdonald.com	UK
4 7	Stacy Hansen	kennethmorris@gmail.com	Canada
5 14	Matthew Lamb	hannahwall@peters-walter.com	Germany
6 15	Amanda Gordon	matthew68@randall.com	UK
7 16	Larry Chapman	zferguson@davis.info	UK
8 19	Maurice Dennis	yzuniga@floyd-mendoza.com	USA
9 20	Brianna Escobar	choibrianna@yahoo.com	Canada
10 21	Scott Lyons	tallen@bradshaw.com	Australia
11 22	Jeremy Hess	samuelbrowning@krueger-thomas.com	Australia
12 23	Lisa Thomas	hannah91@galvan.net	UK
13 24	Jennifer Berger	wesley77@green-miller.com	Australia
14 25	Ashley Freeman	brittanysmac@rushon.com	USA

- Customers Paying Above the Country's Average
 - Uses layered queries to identify high-value consumers.

```

178 -- 3. Find customers who paid more than average in their country
179 Select c.customer_id,c.name,c.country,sum(p.Amount_paid) as amount_paid,
180        (select round(avg(p1.Amount_paid),2)
181         from Payments p1 join Orders o1 on p1.order_id=o1.order_id
182         join Customers c1 on c1.customer_id=o1.customer_id
183         where c1.country=c.country) as Average_by_country
184 from Customers c join Orders o on c.customer_id=o.customer_id
185 join Payments p on o.order_id=p.order_id
186 group by c.customer_id,c.name,c.country
187 having sum(p.Amount_paid)>
188        (select avg(p1.Amount_paid)
189         from Payments p1 join Orders o1 on p1.order_id=o1.order_id
190         join Customers c1 on c1.customer_id=o1.customer_id
191         where c1.country=c.country)

```

83 %

Results Messages					
	customer_id	name	country	amount_paid	Average_by_country
1	7	Stacy Hansen	Canada	5120.00	2558.000000
2	16	Larry Chapman	UK	33440.00	3255.290000
3	19	Maurice Dennis	USA	4000.00	2172.380000
4	20	Brianna Escobar	Canada	9600.00	2558.000000
5	23	Lisa Thomas	UK	11790.00	3255.290000

- Masked Email Addresses for Privacy
 - Showcases data security measures by anonymizing email addresses.

```

192
193 -- 4. Find Customer and mask their emails
194 Select customer_id,[name].country,
195        replace(email,substring(email,2,charindex('@',email)-3),'*****') as masked_email
196 from Customers
197

```

33 %

Results Messages				
	customer_id	name	country	masked_email
1	1	Vanessa Flores	Germany	m*****5@howard.com
2	2	Andrew Rodriguez	Australia	c*****r@yahoo.com
3	3	Emma Davis	UK	k*****a@gmail.com
4	4	Wendy Smith	Canada	j*****r@hotmail.com
5	5	Heather Brown	UK	s*****l@mcdonald.com

- Time Difference Between Order and Payment Dates
 - Uses DATEDIFF() to monitor payment delays and enhance cash flow forecasting.

```

198 -- 5. Calculate the Difference in Days Between Order and Payment Date (Date Functions)
199 Select p.payment_id, p.order_id, o.order_date, p.payment_date,
200        DATEDIFF(day, o.order_date, p.payment_date) as days_to_payment
201 from Orders o JOIN Payments p on o.order_id = p.order_id;
202

```

83 %

Results Messages					
	payment_id	order_id	order_date	payment_date	days_to_payment
1	213	101	2024-04-19	2025-01-02	258
2	214	101	2024-04-19	2025-01-01	257
3	287	101	2024-04-19	2025-01-01	257
4	225	102	2024-04-20	2025-01-01	256
5	210	103	2024-03-23	2025-01-02	285
6	274	103	2024-03-23	2025-01-01	284
7	202	102	2024-03-23	2025-01-02	285

- Consumers Who Have Not Made Any Payments
 - Identifies consumers who placed orders but did not finish payments, which is useful for follow-ups and customer engagement tactics.

```

203 -- 6. Find customers who haven't made any payments
204 Select c.customer_id, c.name from Customers c
205     where not exists (
206         select 1 from Orders o
207             join Payments p on o.order_id = p.order_id
208             where o.customer_id = c.customer_id);
209

```

83 %

Results Messages

	customer_id	name
1	1	Vanessa Flores
2	3	Emma Davis
3	4	Wendy Smith
4	5	Heather Brown
5	6	Theresa Johnson
6	8	Andre Beard
7	9	Joy Gomez
8	12	Thomas Johnson
9	13	Kendra Smith

Each of these queries provides insights that are valuable for business decision-making, improving customer relations, and optimizing financial processes.

Query Optimization Techniques:

Optimising SQL queries is critical for increasing database performance. The key strategies employed are:

- Avoiding Unnecessary Joins: Queries were designed to reduce repetitive data retrieval.
- Filtering Early: WHERE and HAVING clauses were used to reduce processing loads.
- Using EXISTS instead of IN: EXISTS is typically more efficient for subqueries.
- Avoiding SELECT: Only necessary columns were chosen to reduce memory usage.

These enhancements ensured that searches ran efficiently even as data volumes rose.

Conclusion:

This work gave a thorough understanding of database design, data importation, and SQL query execution. The well-structured OnlineShoppingDB maintains data integrity, and the queries yield useful business insights.

The key lessons from this research include:

- Database normalization ensured that the data structure was scalable and not redundant.
- Data integrity and validation were achieved by utilizing constraints and foreign keys to prevent discrepancies.
- Advanced SQL Techniques: Executed subqueries, joins, and system operations efficiently.
- Query Optimization: Used indexing and filtering techniques to boost performance.
- Business Relevance: Each query delivers useful information for making decisions in an online commerce environment.